CNENGR202 Instrumentation and Control

Part 3 L8 – Analogue to Digital Conversion

Prof. Zhang Liwei 张立伟 教授

Email: lwzhang@bjtu.edu.cn
Working Room: SX702

School of Electronics and Information Engineering
Beijing Jiaotong University

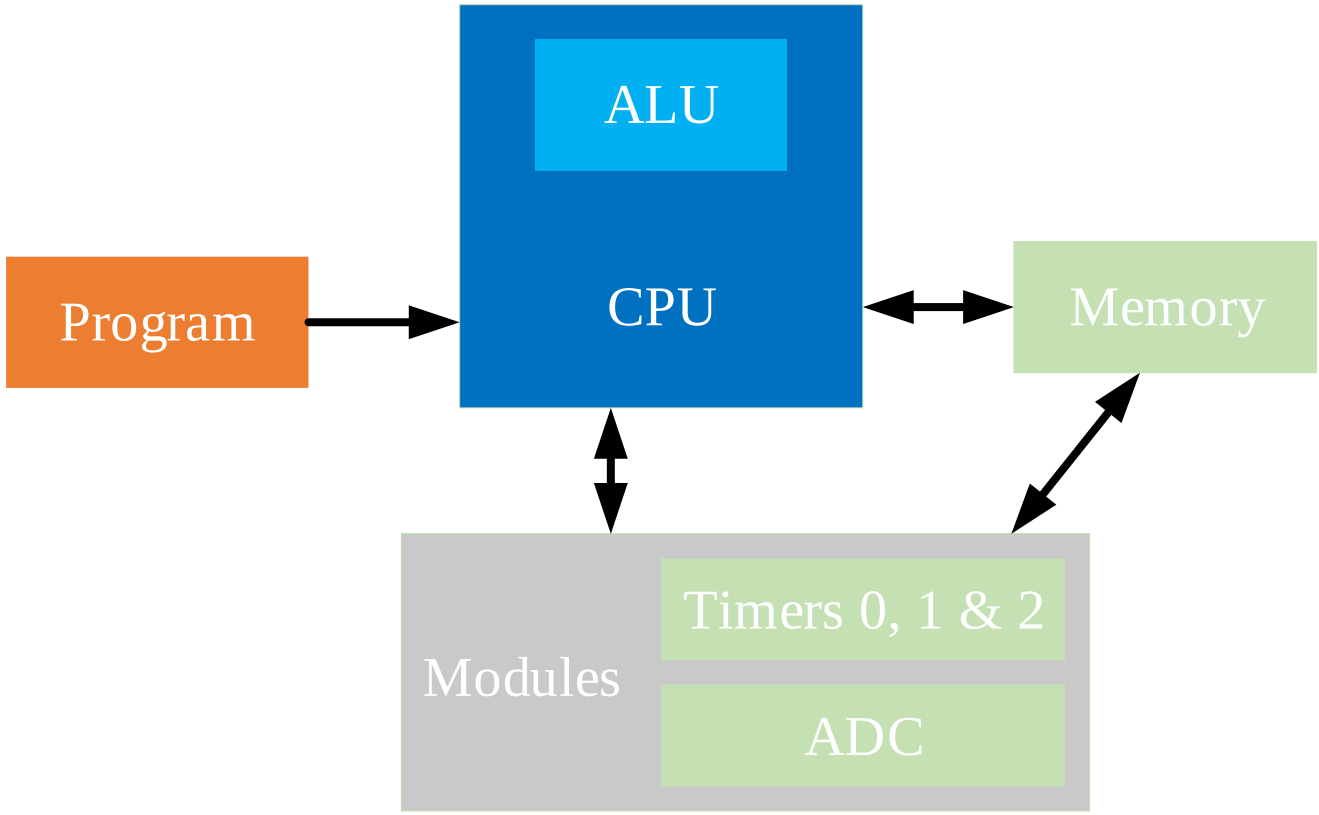
Recap

- React to external devices
- Waveform generator – change the frequencies
- PWM – change duty cycle
- Phase corrected PWM

Analogue to Digital Conversion

- Many sensors have analogue outputs
  - Strain
  - Light
  - Pressure
  - Temperature
- Often processing is needed
  - Analogue only electronics for calculation is complex, unreliable and hard to reprogram
  - To process analogue signals digitally we must convert them

Modules

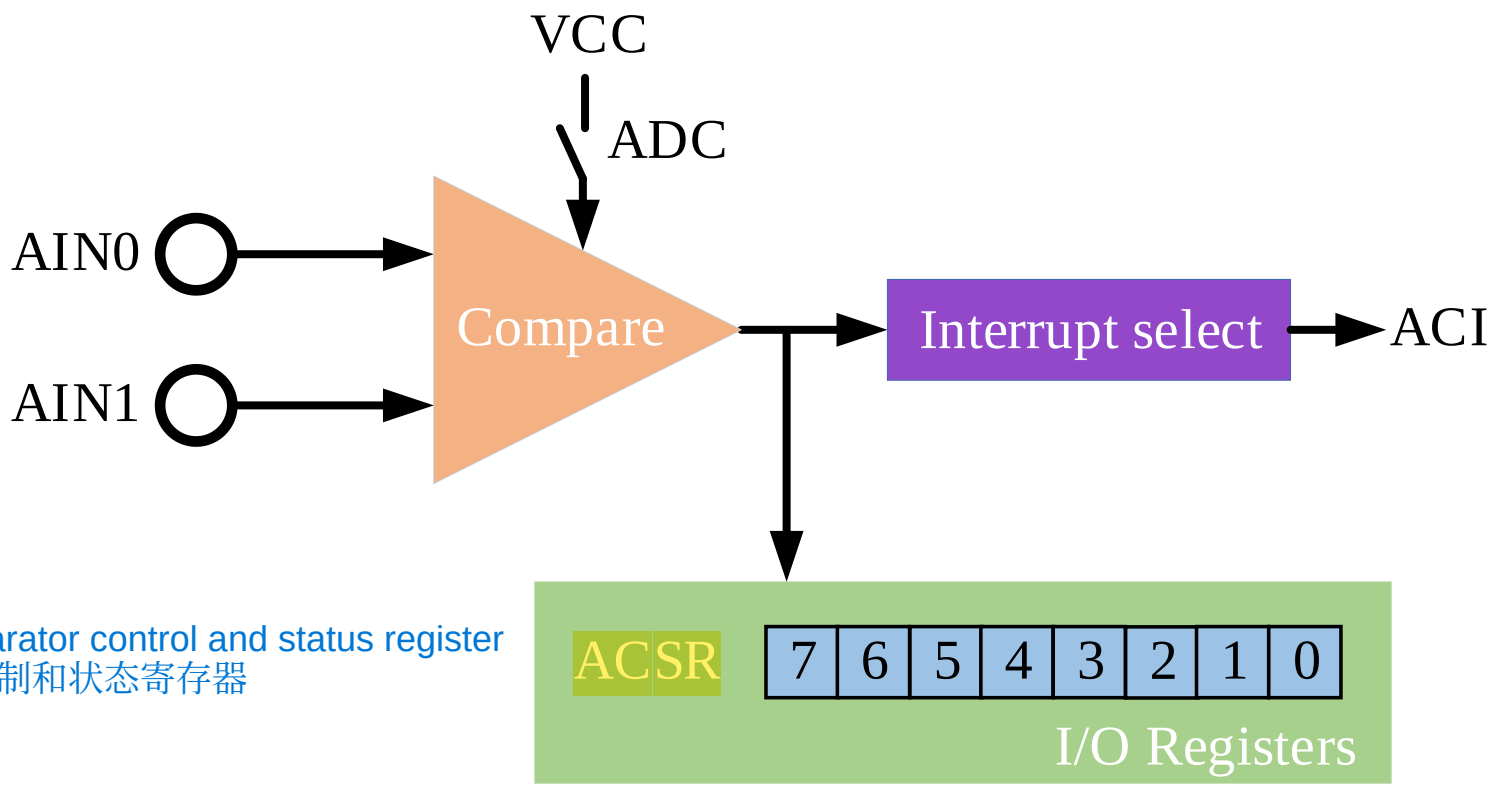


Analogue to digital conversion

- Two ways are implemented in these chips
  - Compare
  - Proper ADC

Comparator

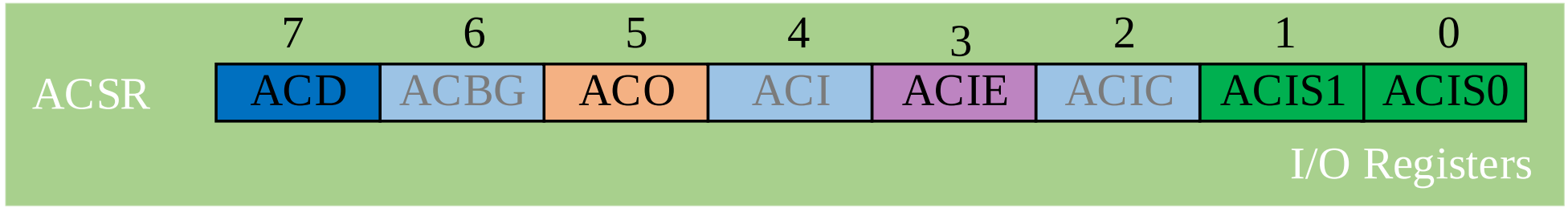
Comparator



analog comparator control and status register
模拟比较器控制和状态寄存器

Control register (ACSR)

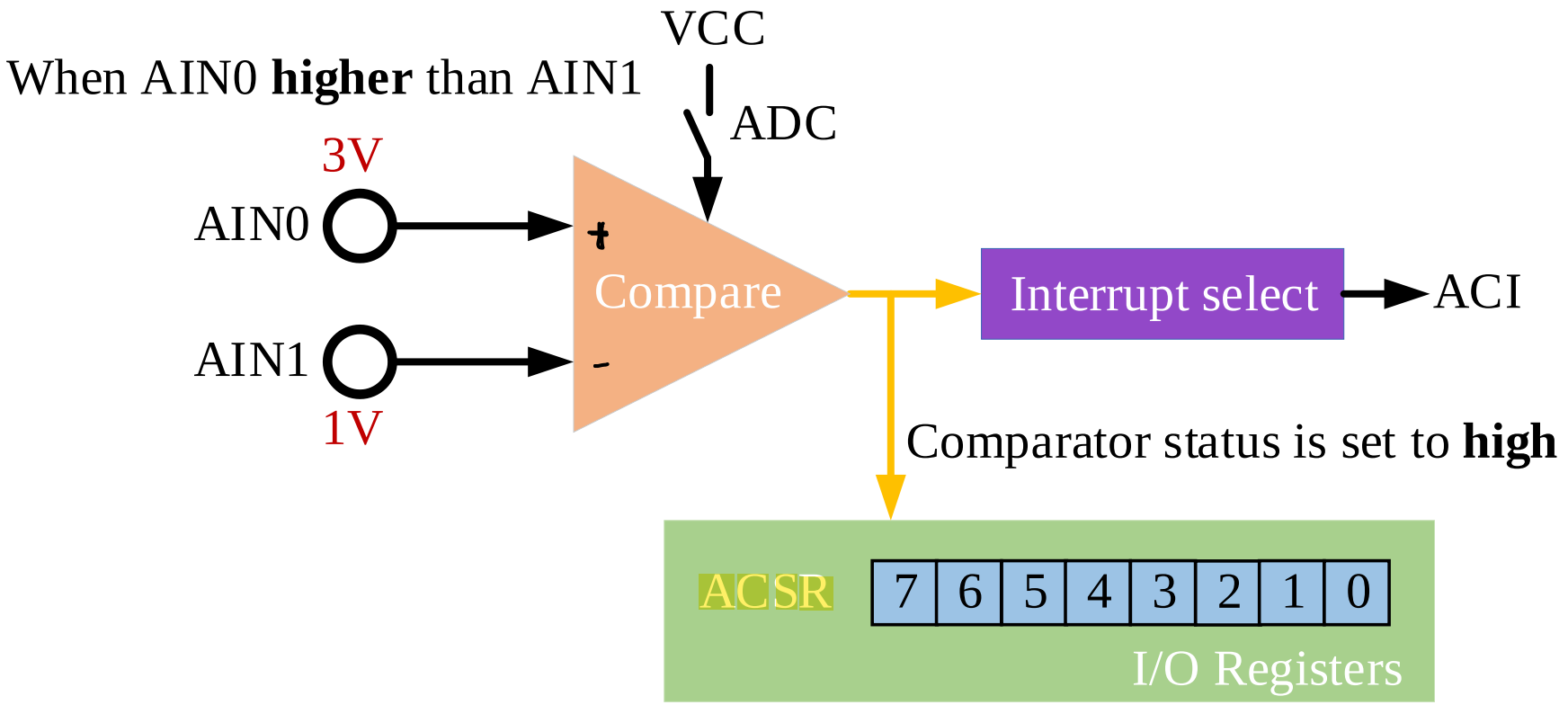
- ACD disable the comparator
- ACO represents the current status of the comparator
- ACIE enables the interrupt (vector ACIaddr)
- ACISn choose when to interrupt



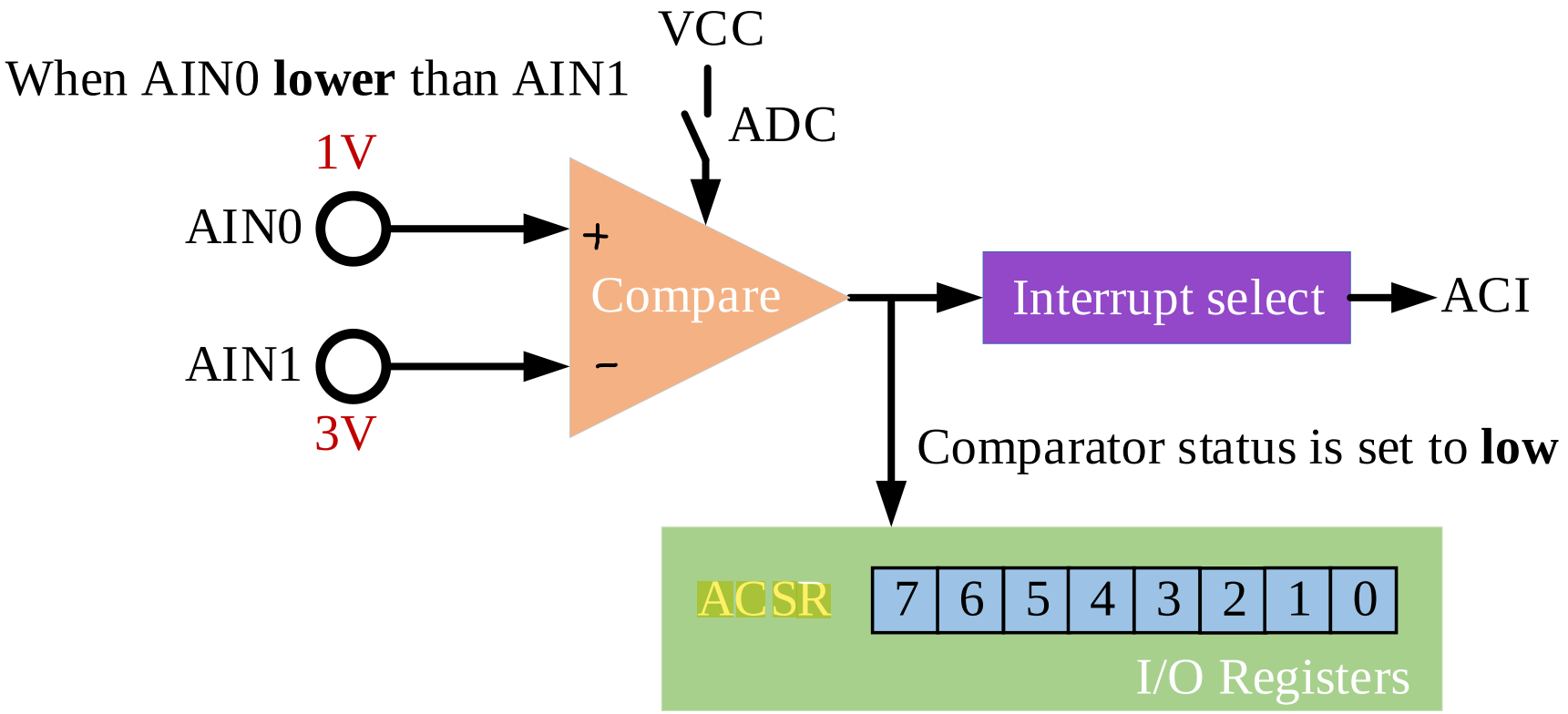
ACIS1/ACIS0 Settings

ACIS1	ACIS0	Interrupt Mode
0	0	Comparator Interrupt on Output Toggle
0	1	Reserved
1	0	Comparator Interrupt on Falling Output Edge
1	1	Comparator Interrupt on Rising Output Edge

Comparator

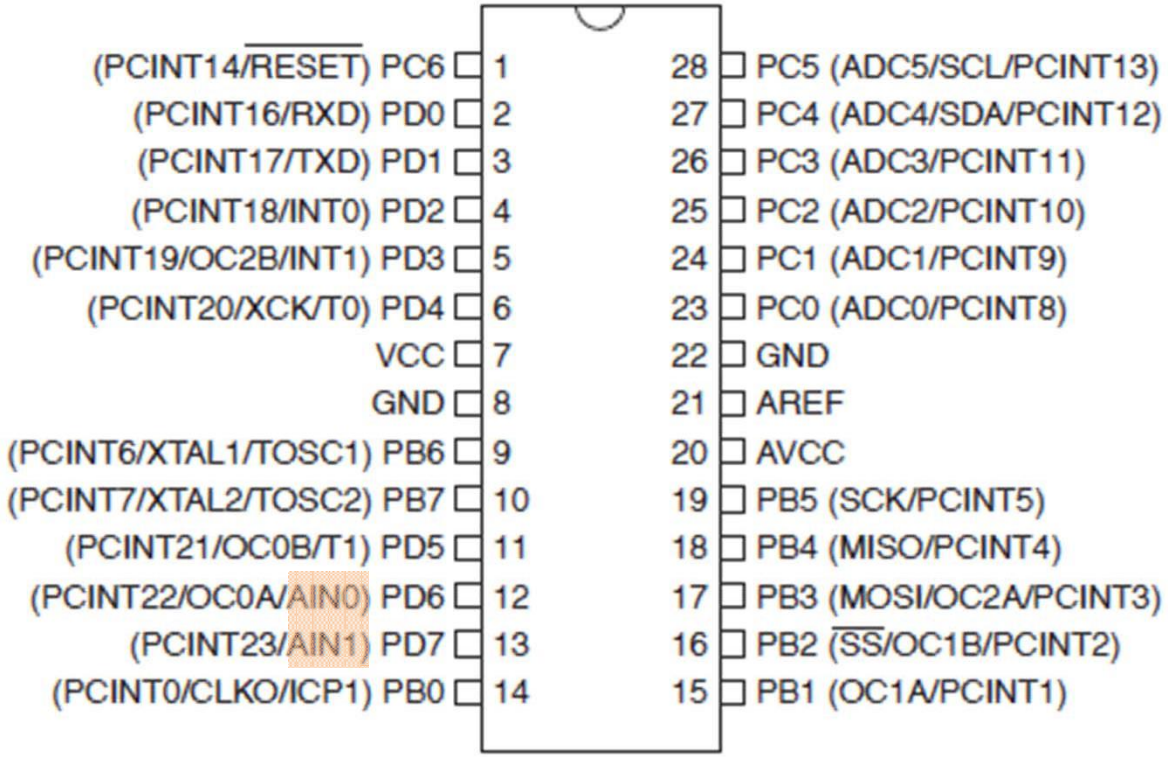

When AIN0 **higher** than AIN1
VCC
ADC
3V
AIN0
AIN1
1V
+
-
Compare
Interrupt select
ACI
Comparator status is set to **high**
ACSR
7
6
5
4
3
2
1
0
I/O Registers

Comparator


When AIN0 **lower** than AIN1
VCC
ADC
1V
AIN0
+
Compare
AIN1
–
3V
Interrupt select
ACI
Comparator status is set to **low**
ACSR
7
6
5
4
3
2
1
0
I/O Registers

Analogue Comparator External Connections

- Just two pins



ADC (Analogue to Digital Converter)

Analogue to Digital Converter

- **The ADC on the 328P we are using has the following properties:**
  - 10 - bit resolution;
  - linear successive approximation;
  - Six input channels;
  - Single or continuous conversion;
  - Conversion complete flag/interrupt;
  - Selectable ADC clock.
  - Sample and hold for rapid changing signals

ADC in Microcontrollers

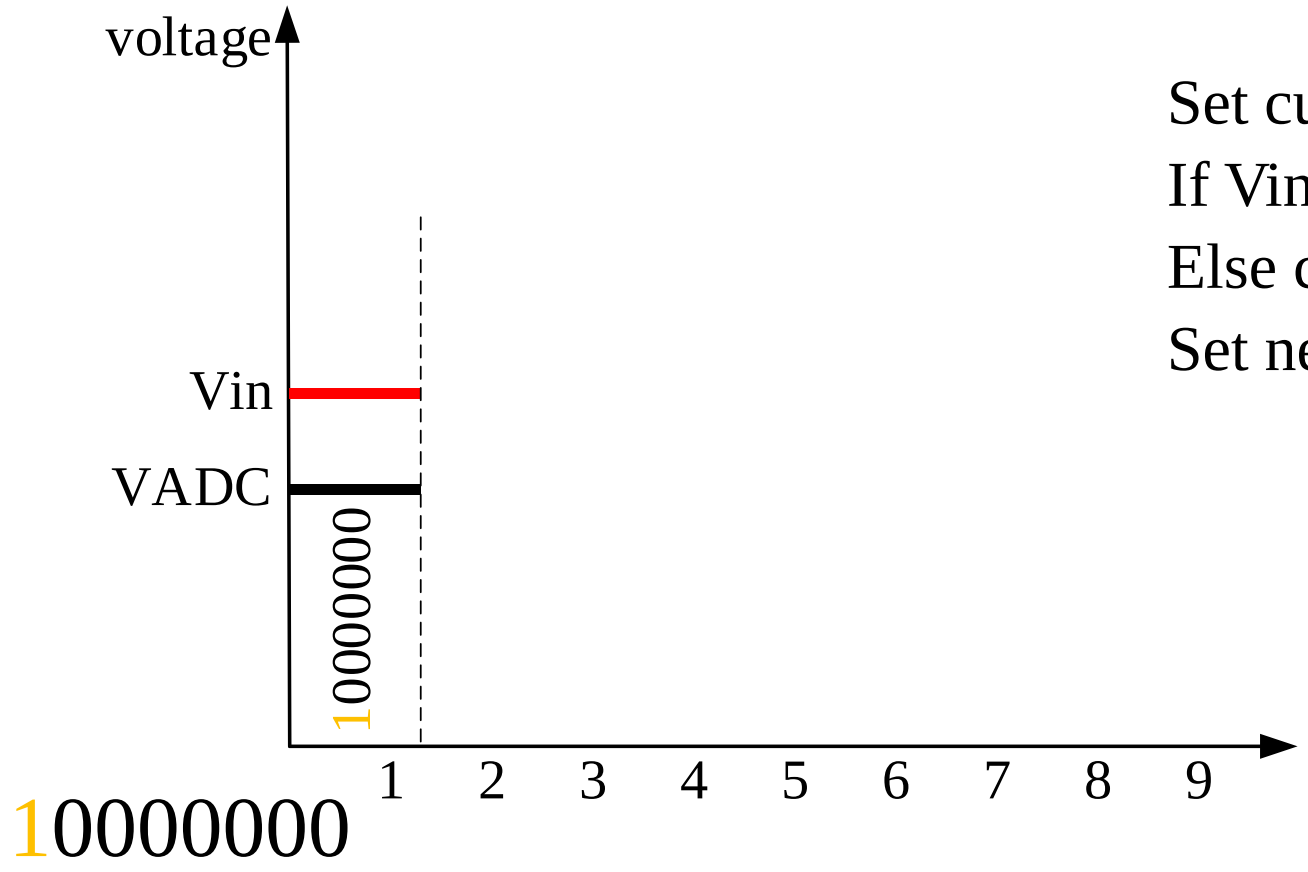
- Important performance parameters in A - D converters
  - Resolution – number of bits n gives 2^n levels
  - Speed – conversion rate – samples / s
  - Monotonicity
    - Does the digital output increase with analogue input increase? (vice versa)
  - Linearity
    - How close is the actual output from the ideal output?

Successive Approximation ADC

Problem: How do we find the best quantised level for a particular analogue input value?

- Start with most significant bit
  1. Set current bit
  2. If Vin > VADC, leave current bit, otherwise clear it
- Repeat step 1. for ***next*** most significant bit

Successive Approximation ADC



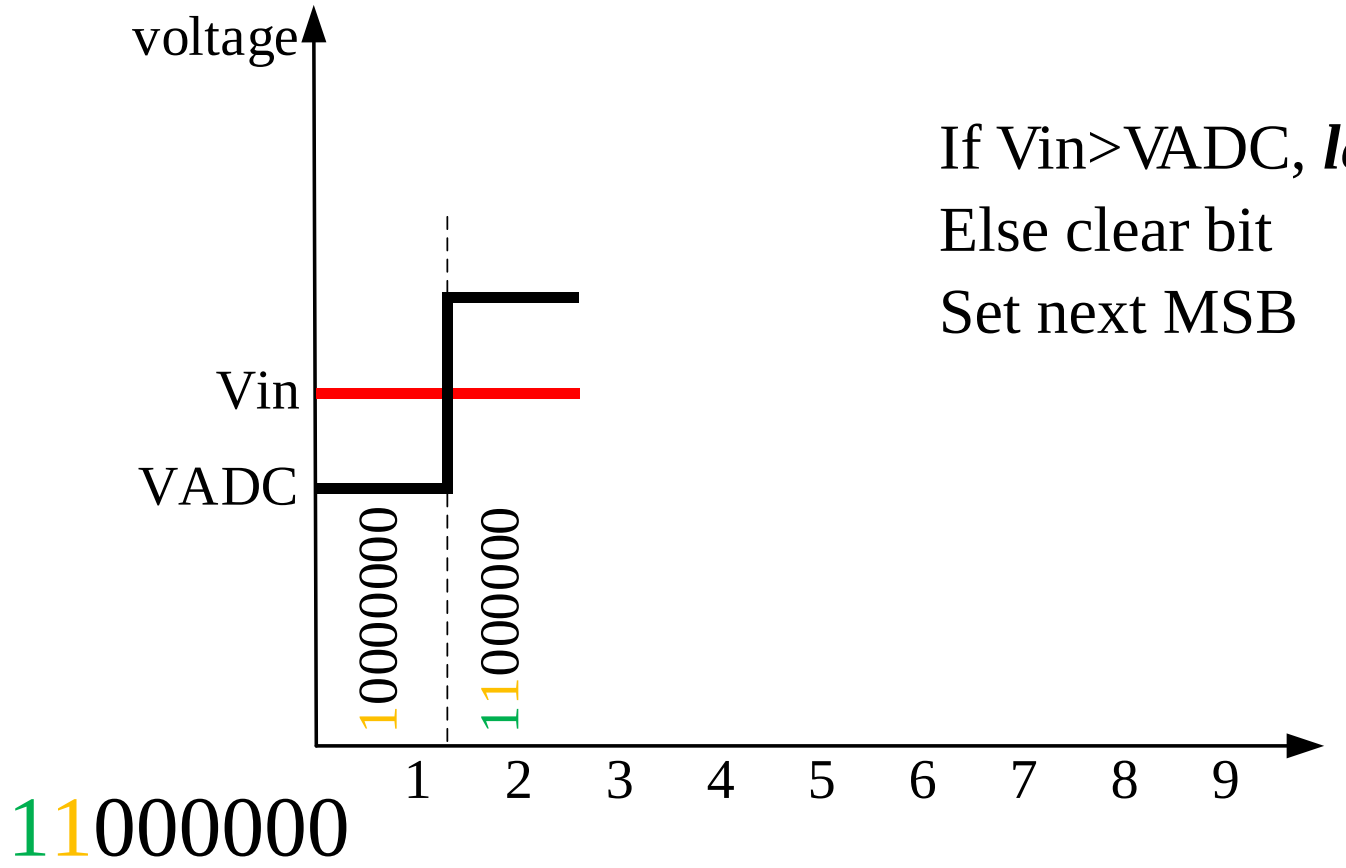
Set current bit

If Vin>VADC, ***leave*** current bit

Else clear bit

Set next MSB

Successive Approximation ADC

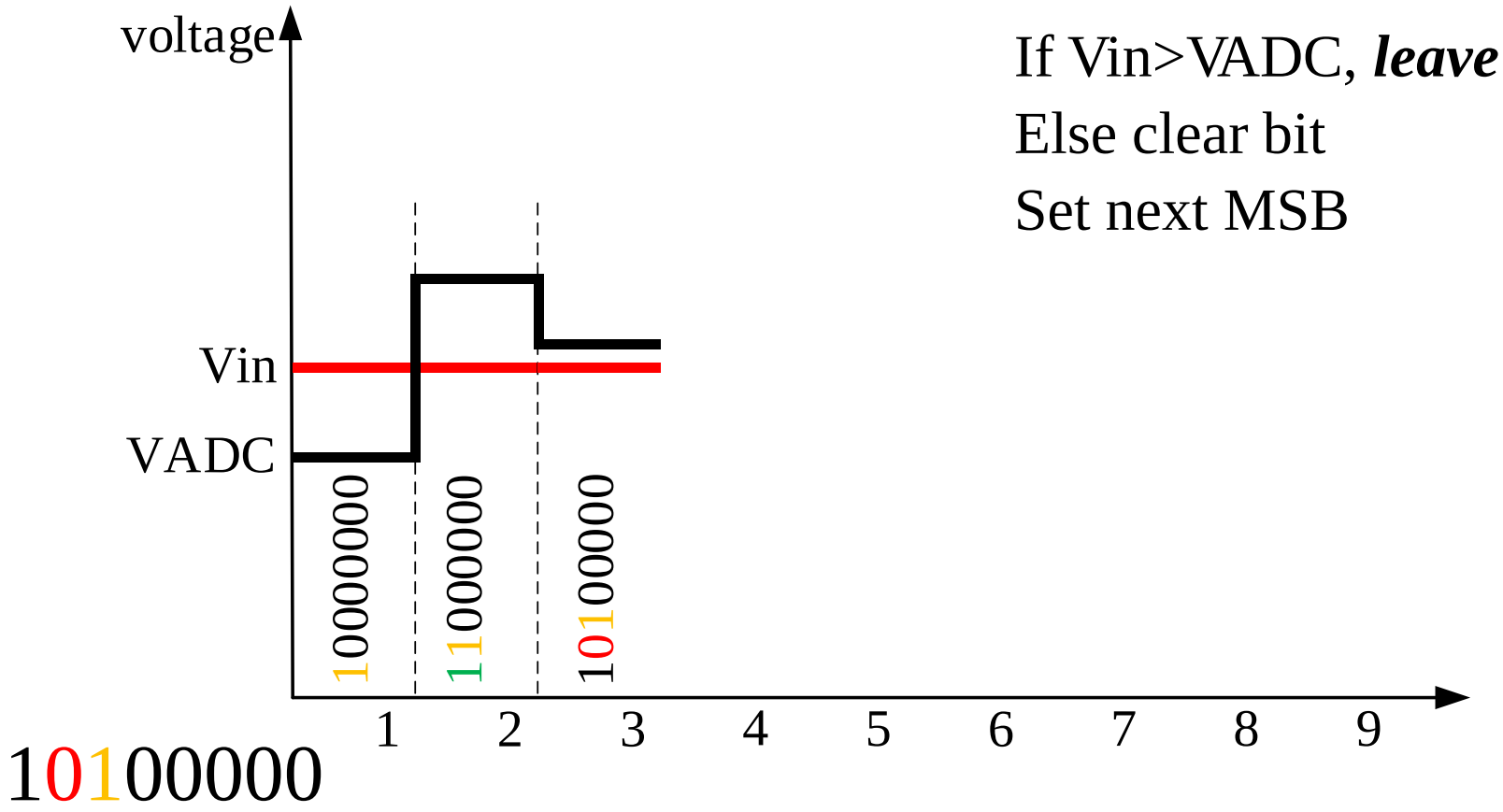


If Vin>VADC, ***leave*** current bit

Else clear bit

Set next MSB

Successive Approximation ADC

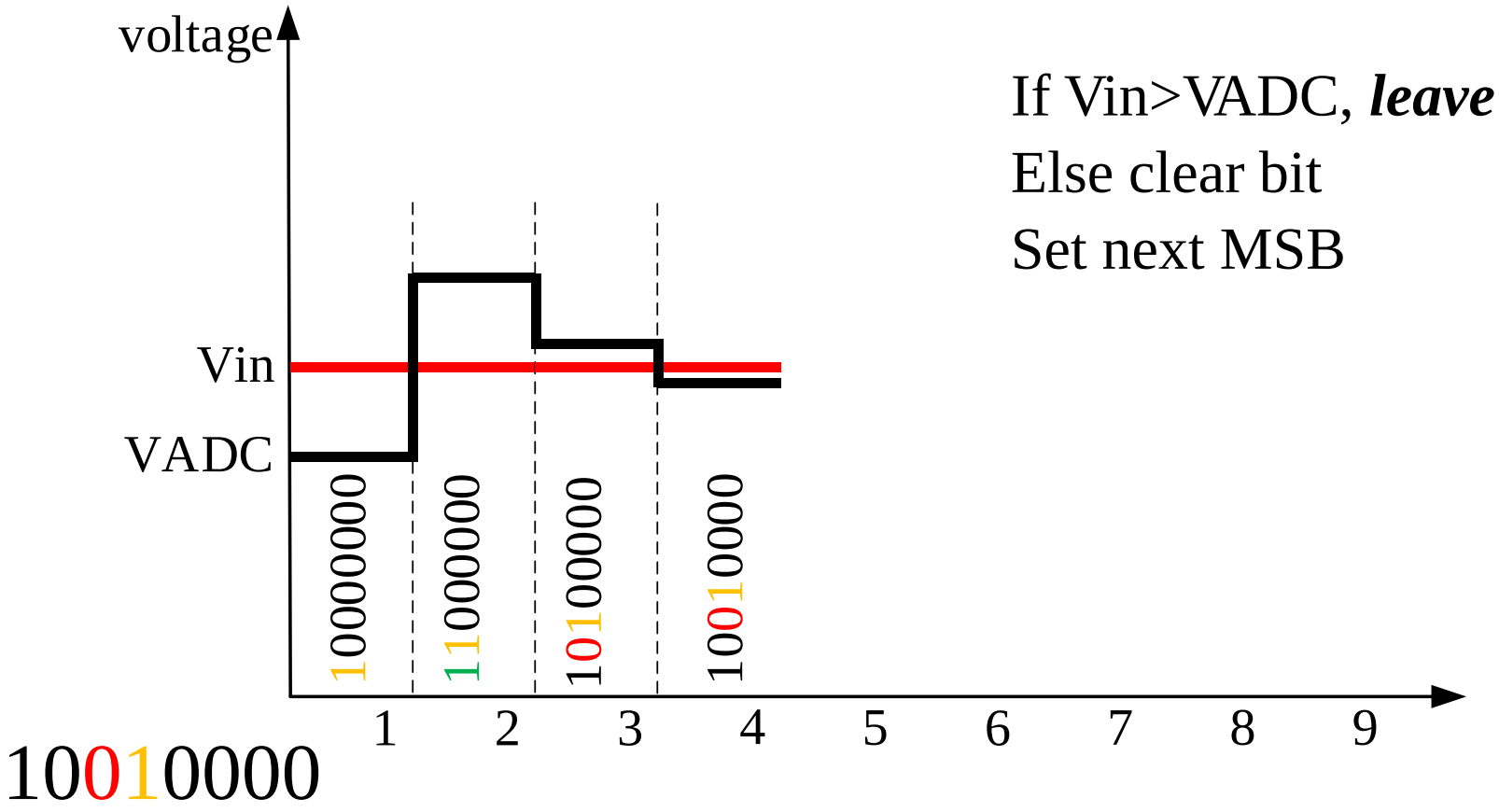


If Vin>VADC, ***leave*** current bit

Else clear bit

Set next MSB

Successive Approximation ADC

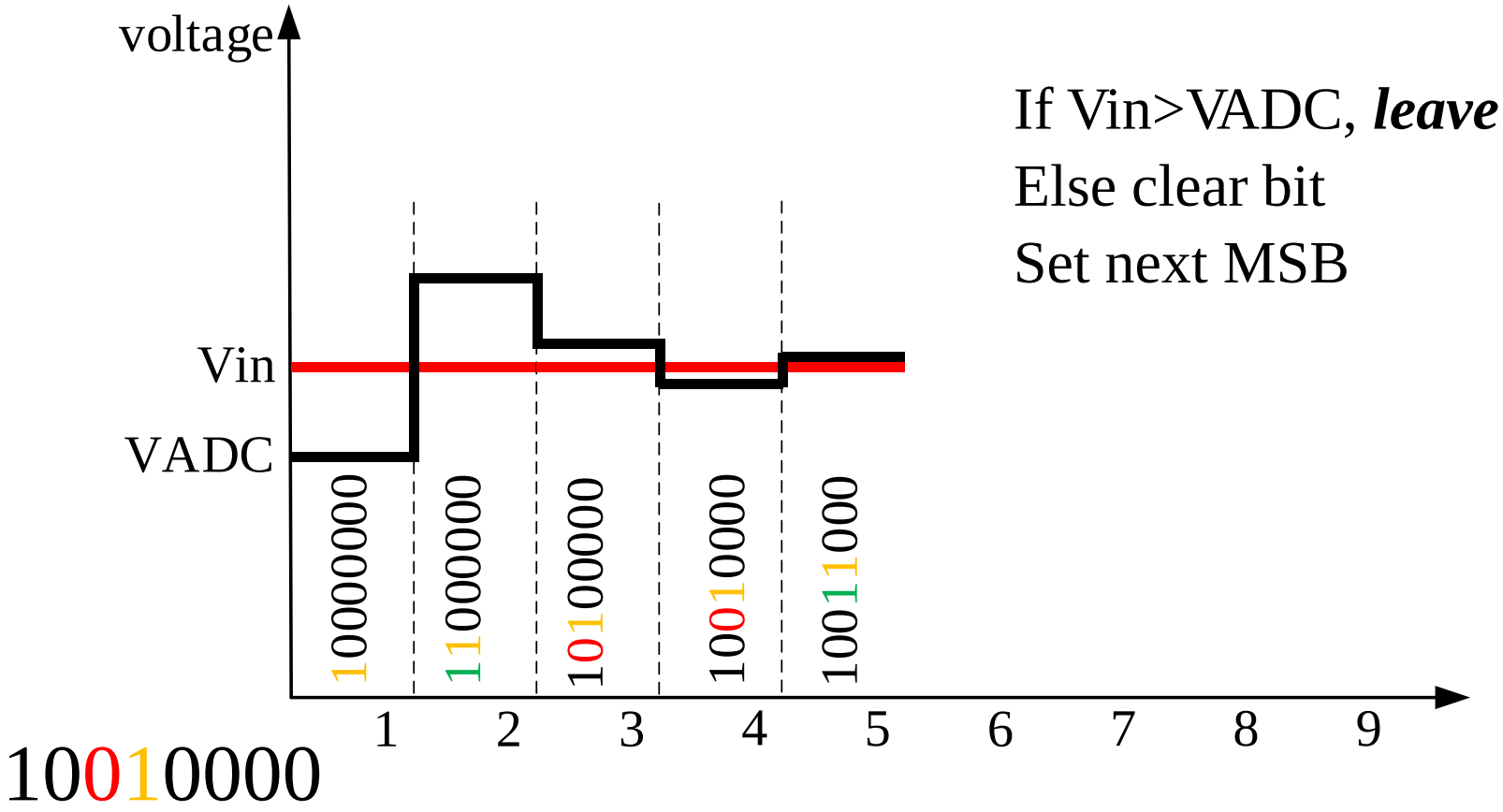


If Vin>VADC, ***leave*** current bit

Else clear bit

Set next MSB

Successive Approximation ADC

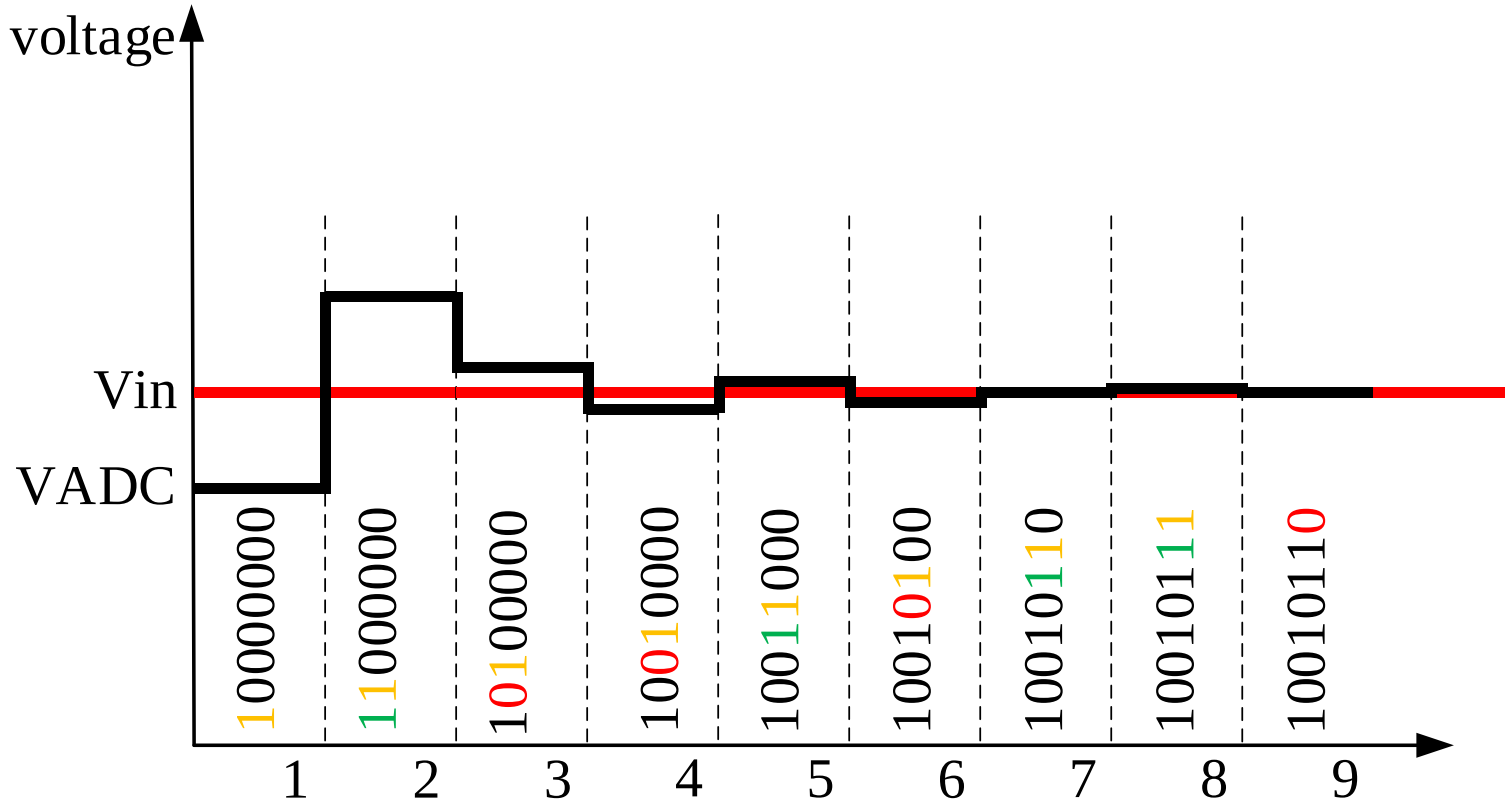


If Vin>VADC, ***leave*** current bit

Else clear bit

Set next MSB

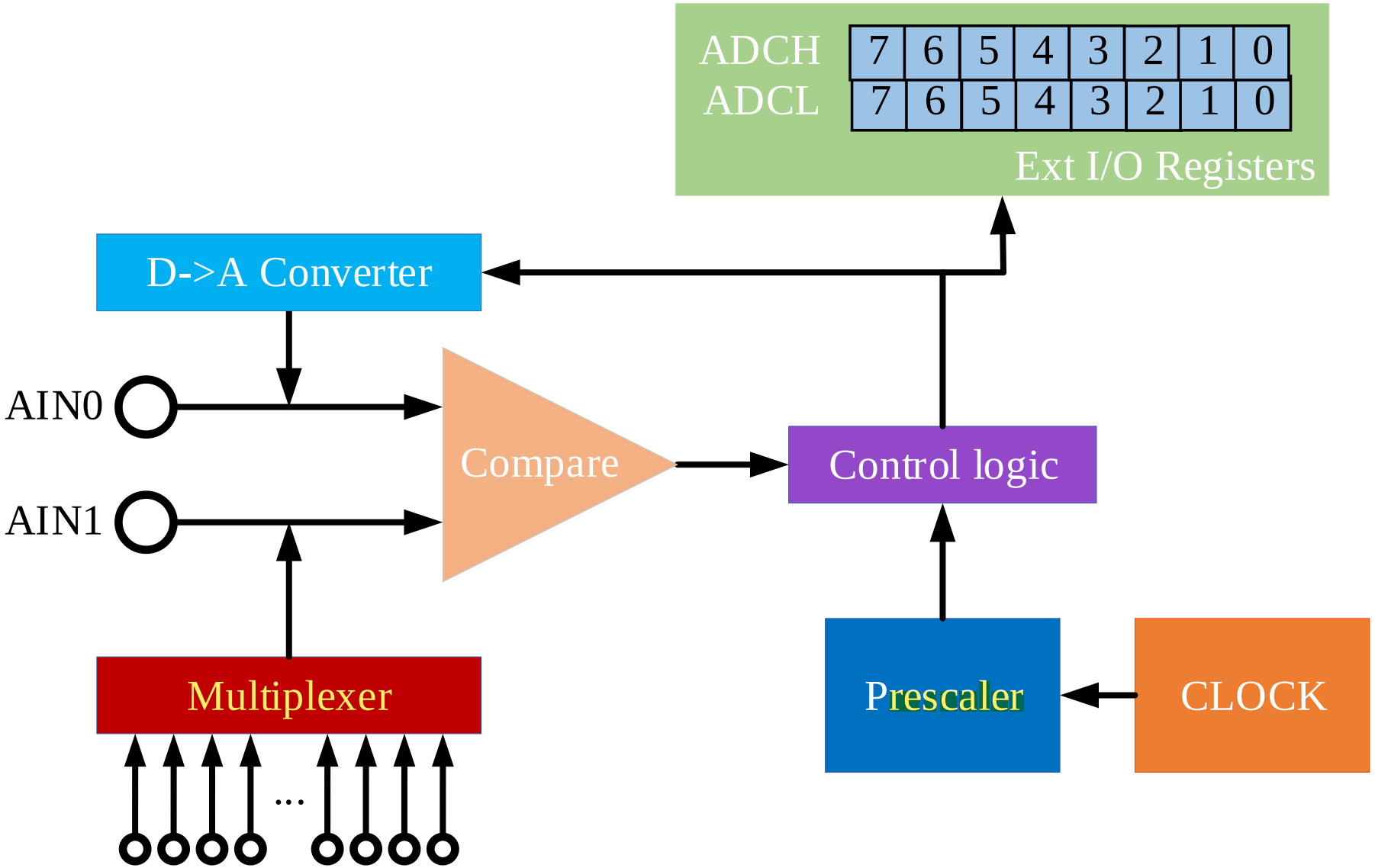
Successive Approximation ADC



What does this look like?

- Successive approximation ADC is:
  - part DAC (digital to analogue converter)
  - part comparator

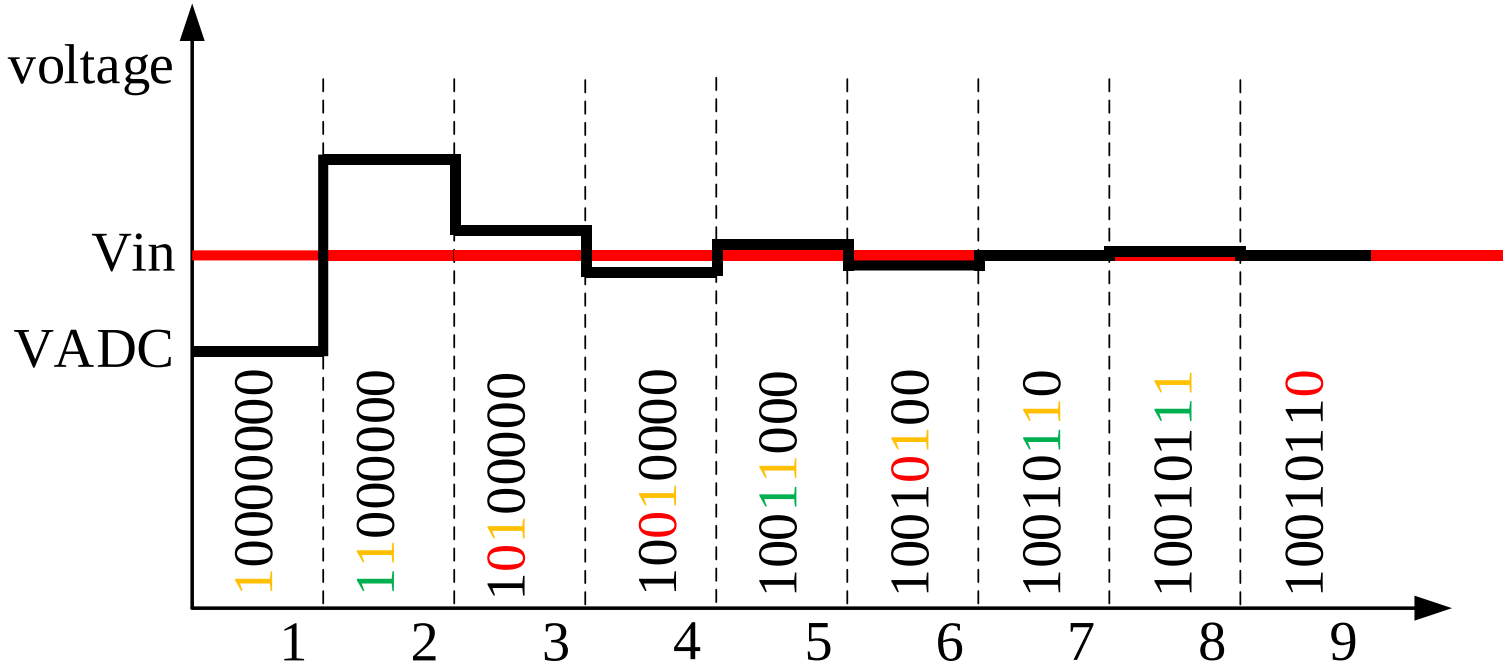
A - >D Converter



Conversion time

- Cycle speed is dependent on DAC settling time.
- The time to convert is n + 1 cycles for n - bit accuracy.

- Conversion on our chip takes 13 cycles
  - Additional delay/overheads caused by control logic



Prescaler

- To give ADC time to achieve 10bit accuracy need clock between 10kHz and 200kHz
- 16MHz/128 = 125kHz

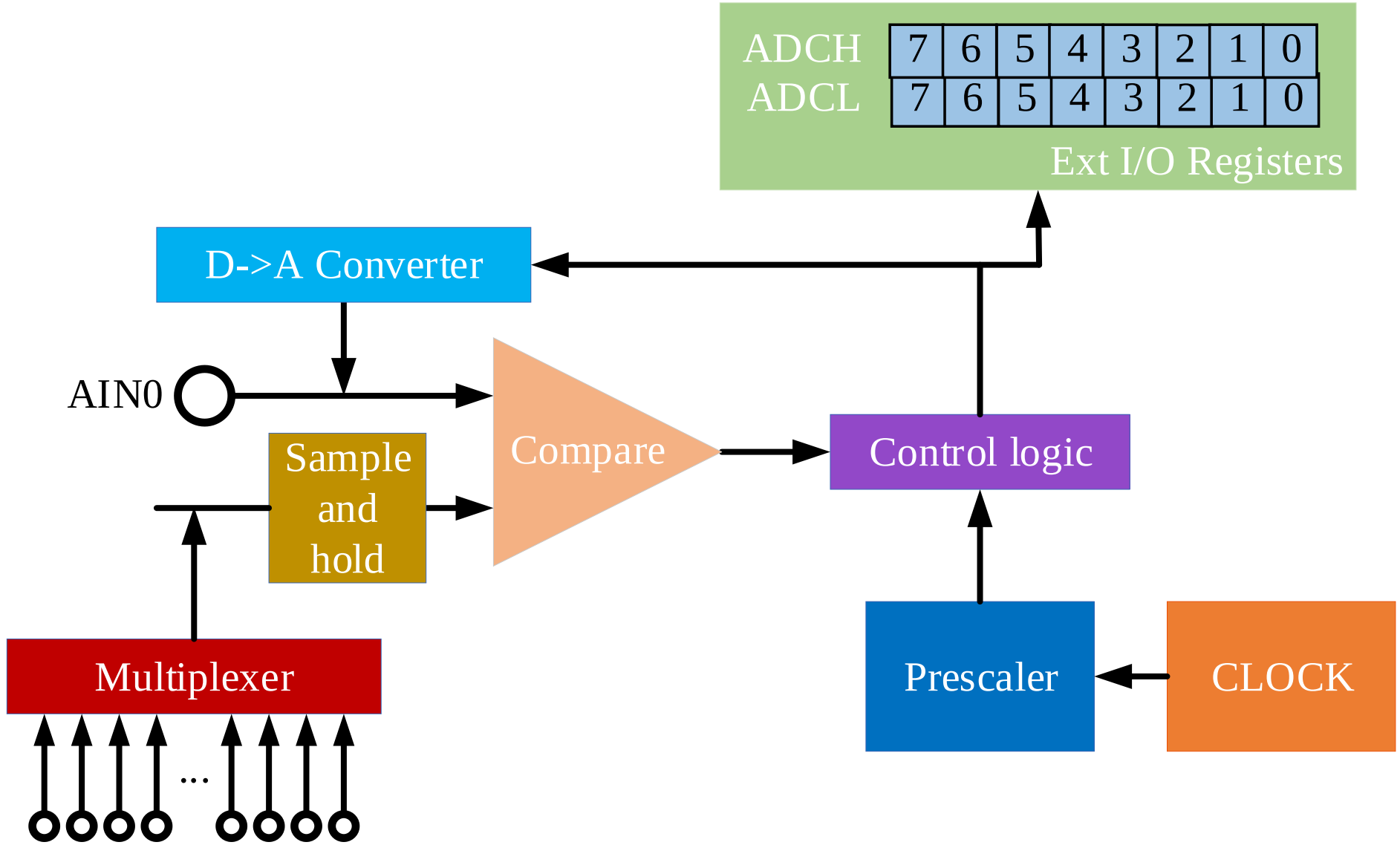
ADC Prescaler Selections

ADPS2	ADPS1	ADPS0	Division Factor
0	0	0	2
0	0	1	2
0	1	0	4
0	1	1	8
1	0	0	16
1	0	1	32
1	1	0	64
1	1	1	128

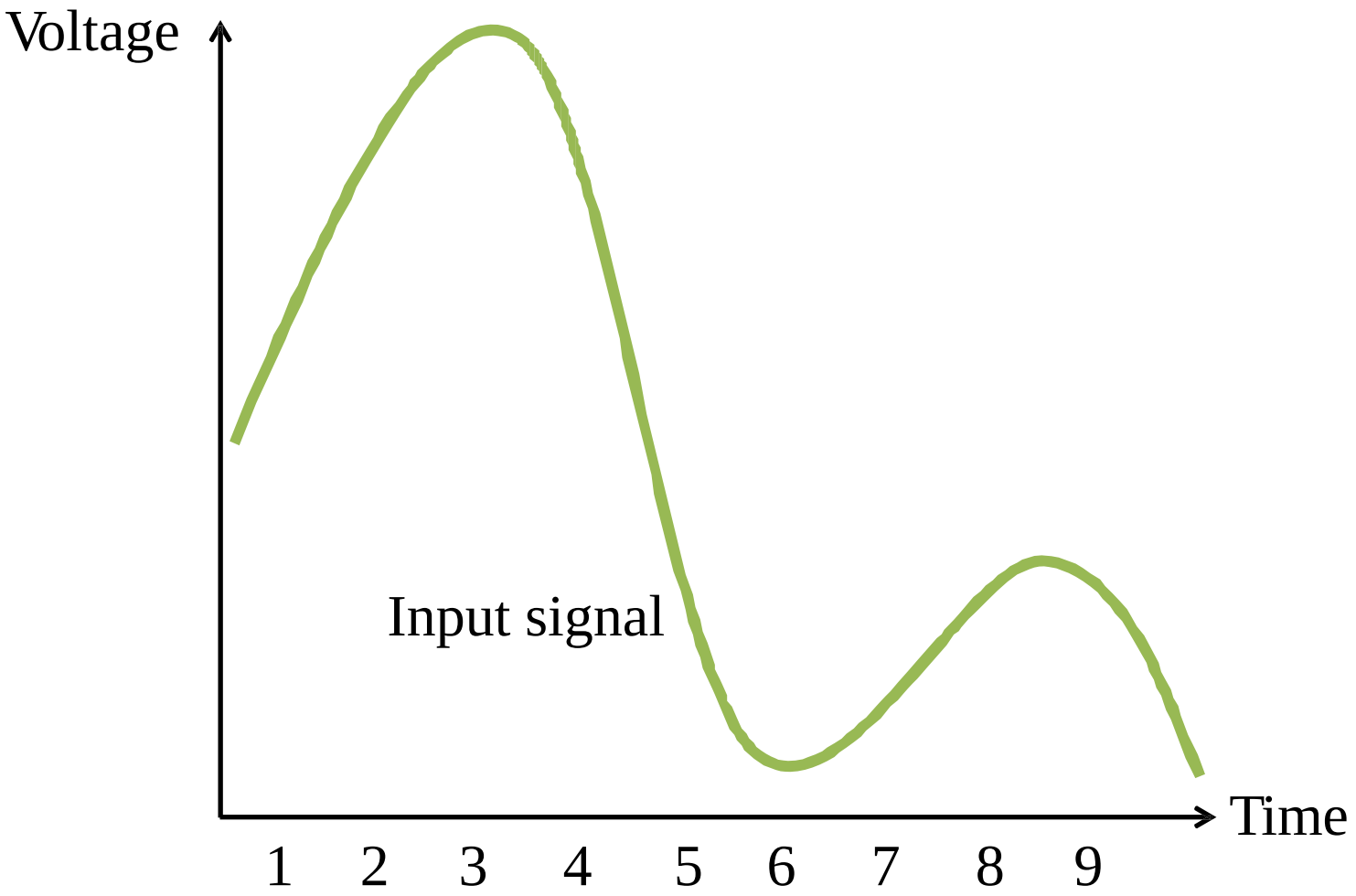
Sample and hold

- Conversion takes a finite amount of time
- Voltage must remain stable
- Useful to be able to measure non static voltages
- Sample and hold circuit.
  - For a single ADC conversion.
    - Samples voltage
    - Holds voltage
- Sample and hold ensures that the input does not change during conversion

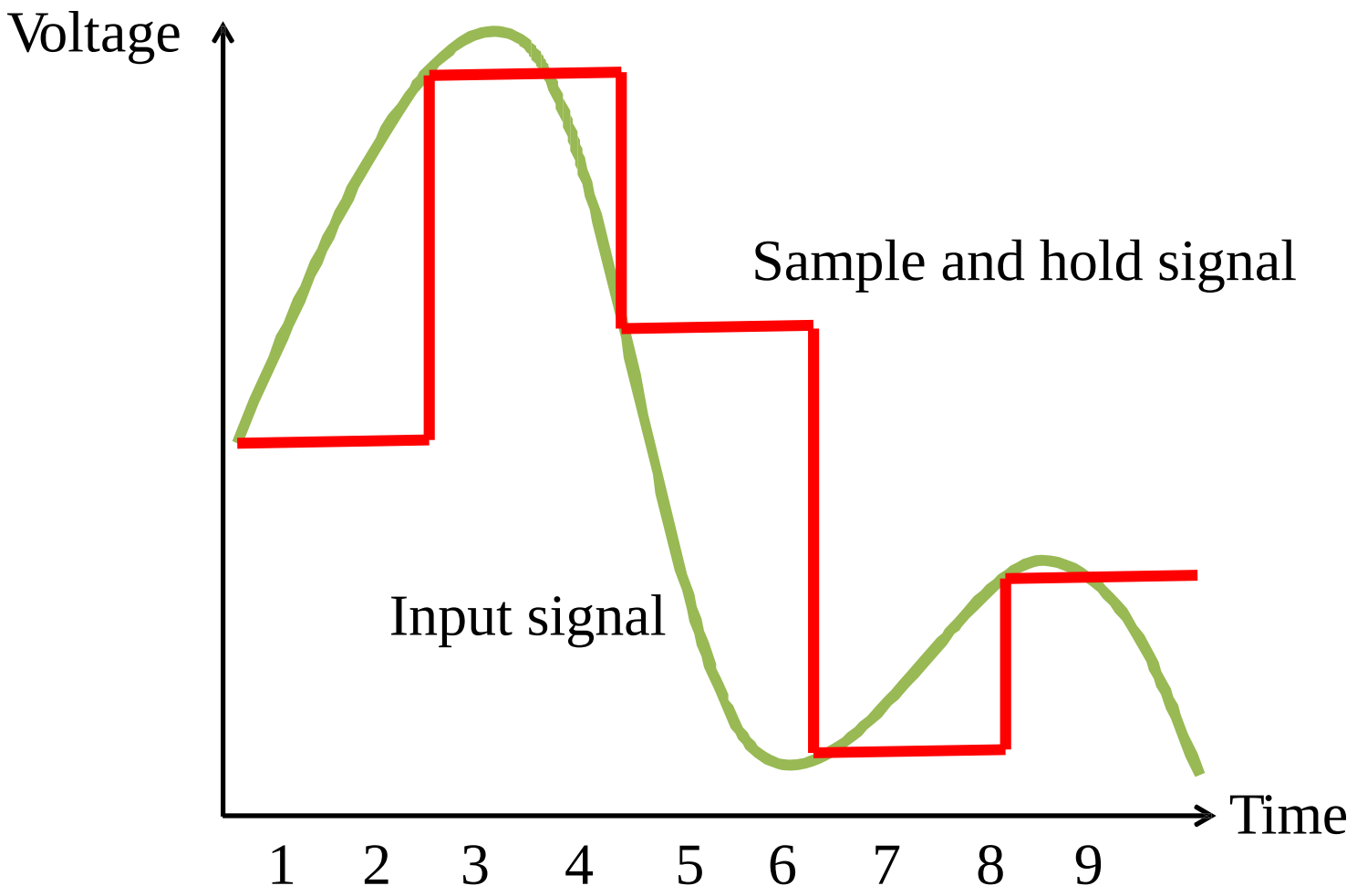
Sample and hold



Sample and hold



Sample and hold

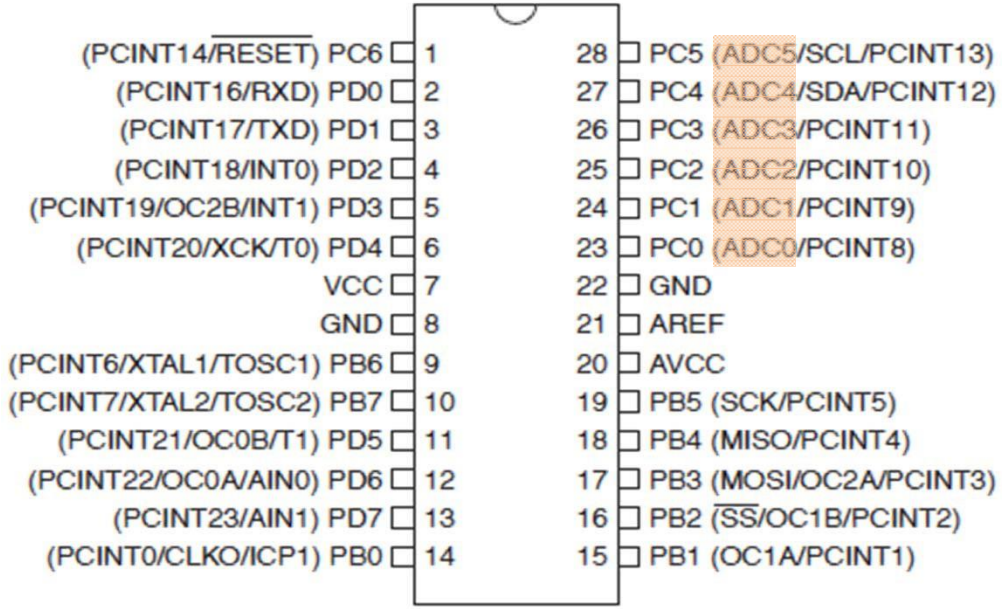


Dynamic Range

- Note that there are voltage limits:
  - Minimum = 0x00 on conversion (ground).
  - Maximum = 0x3FF on conversion (ADC power)
- Therefore, condition your input voltage according to these limits to maximise your use of the ADC ***dynamic range***.

ADC External Connections

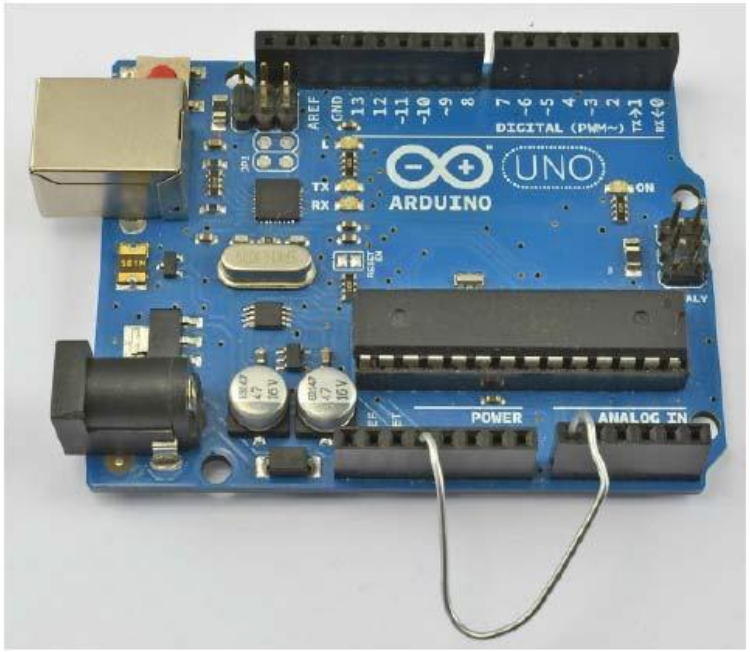
- 6 pins can be sampled ADC0 - 5
- Voltage reference from AREF





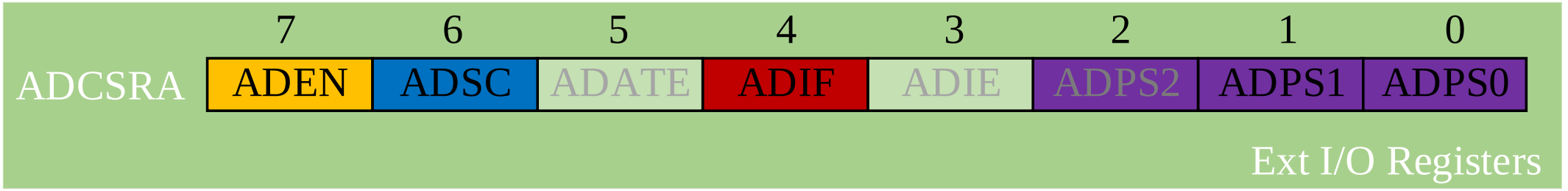
ADC Programming

- Basic usage of ADC is simple
- Simple example: read in the voltage from the 3.3 V supply from Arduino



ADC状态和控制寄存器

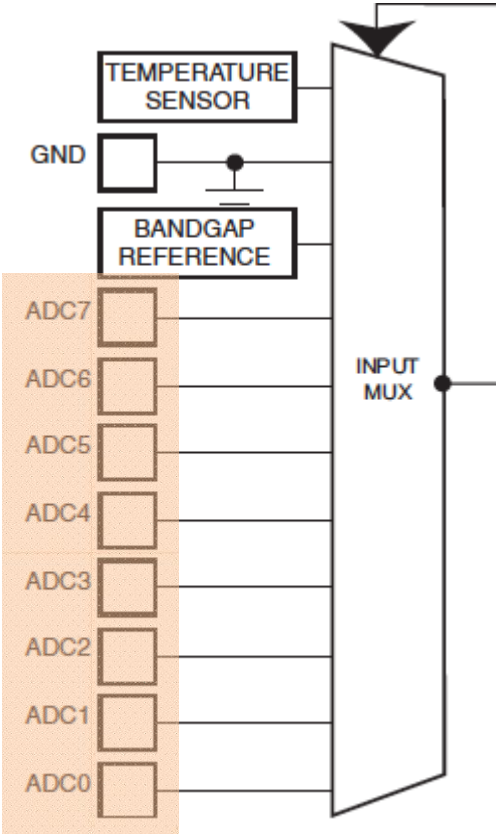
ADC Status and Control Register



- It is best to initialise this register in the setup, as was done for the timer counters
- ADEN enables the ADC
- ADSC starts a conversion
- ADIF is toggled when conversion is complete
- ADPSn sets the divider for the ADC prescaler (just like the timer counters

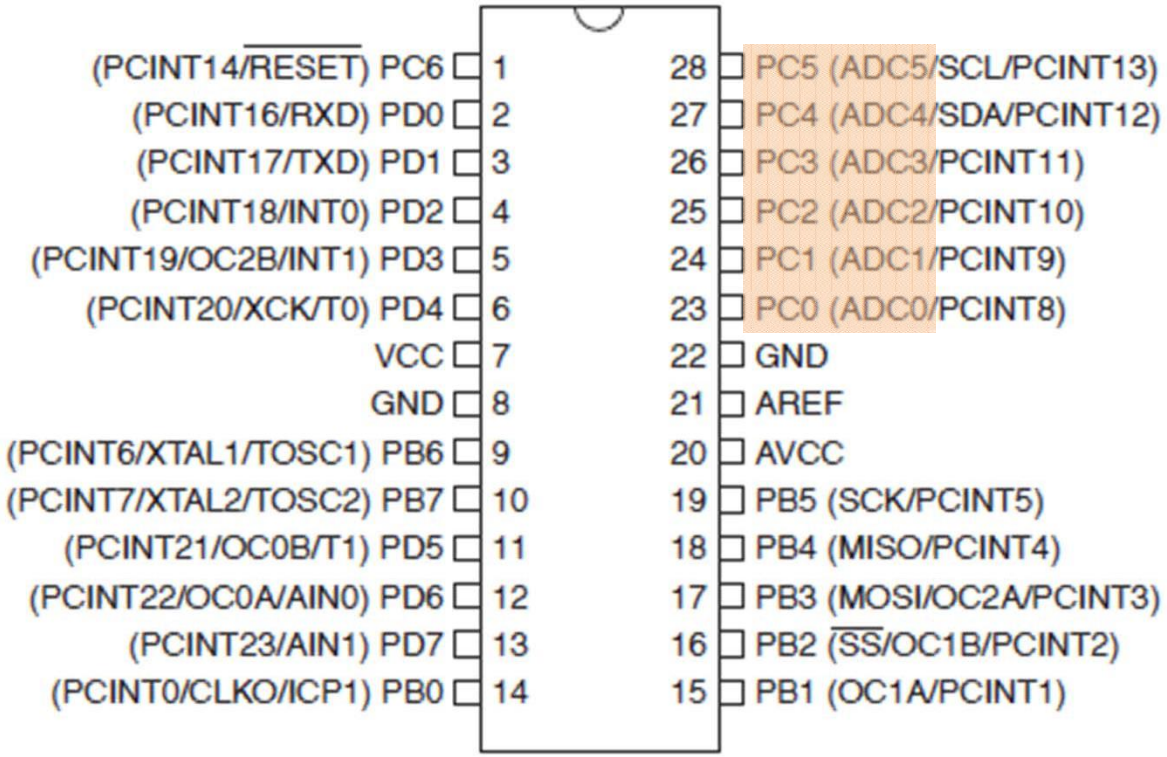
Input Mux

- There are 8 pins which can be used as inputs for the ADC
- By cycling through them you can measure 8 different things externally
- A temperature sensor is also included on chip called ADC8 in the following table
- A different package (with more pins) allows access to ADC6 and ADC7
- 328p allows ADC0 - >ADC5

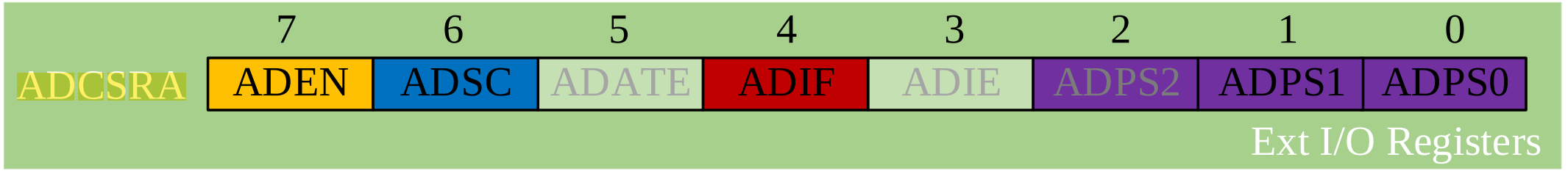


ADC External Connections

- 6 pins



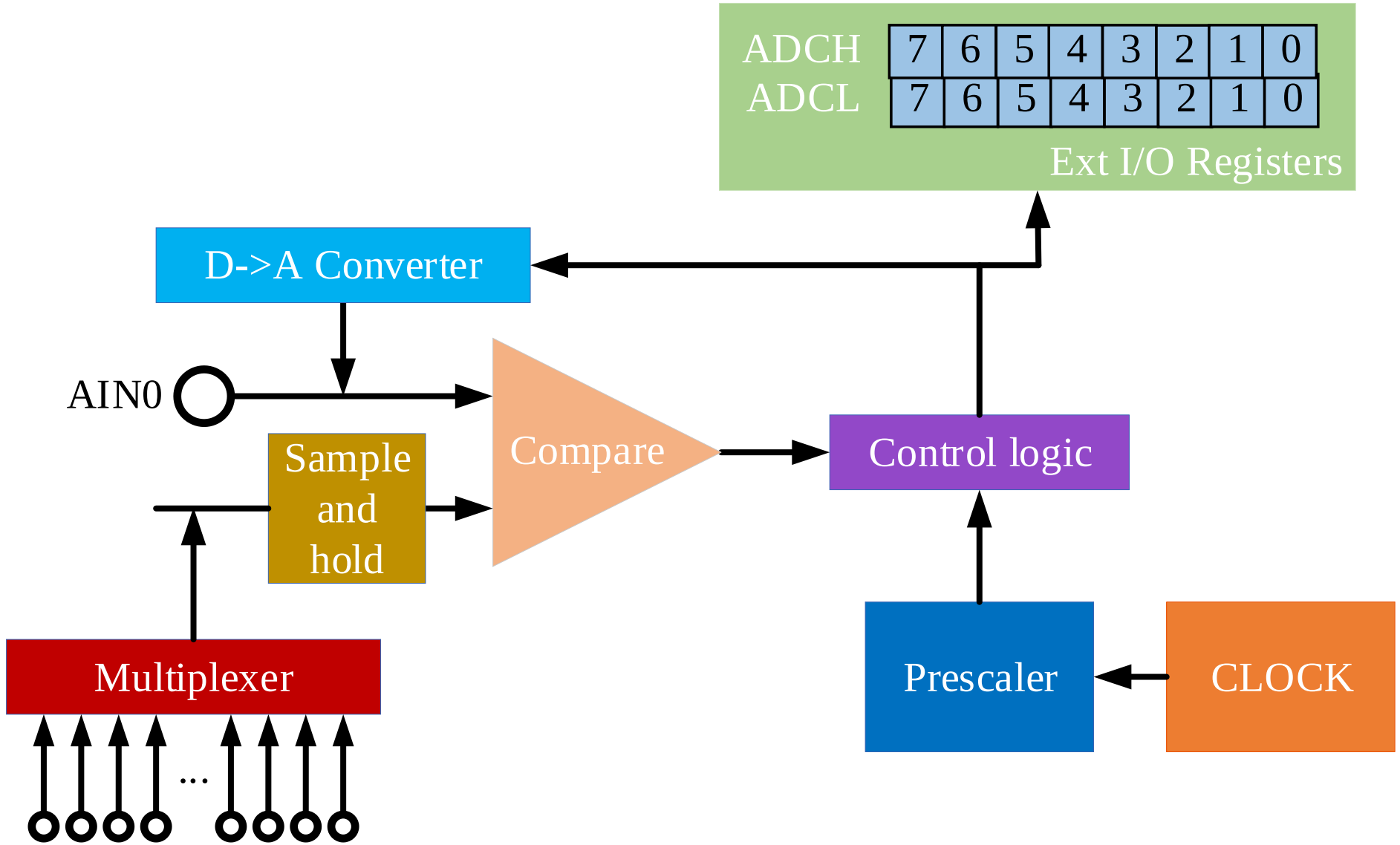
ADC Input Channel Select



Input Channel Selections

MUX3.0	Single Ended Input
0000	ADC0
0001	ADC1
0010	ADC2
0011	ADC3
0100	ADC4
0101	ADC5
0110	ADC6
0111	ADC7
1000	ADC8
1001	Reserved
1010	Reserved
1011	Reserved
1100	Reserved
1110	1.1V(VBB)
1111	0V(GND)

After conversion?



When conversion is complete

	7	6	5	4	3	2	1	0
ADCH	-	-	-	-	-	-	ADC9	ADC8
ADCL	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADC1	ADC0

Ext I/O Registers

- Upon conversion completion, the result is written into an 10 - bit register **ADCH and ADCL**.

Left Adjustment

	7	6	5	4	3	2	1	0
ADMUX	REFS1	REFS0	ADLAR	-	MUX3	MUX2	MUX1	MUX0

Ext I/O Registers

ADLAR = 0

	7	6	5	4	3	2	1	0
ADCH	-	-	-	-	-	-	ADC9	ADC8
ADCL	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADC1	ADC0

Ext I/O Registers

ADLAR = 1

	7	6	5	4	3	2	1	0
ADCH	ADC9	ADC8	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2
ADCL	ADC1	ADC0						

Ext I/O Registers

All of the most significant bits are grouped in ADCH so ADC can easily be used as 8 bit

Reading the ADC

- Typical variable assignment

  int low = ADCL;

  int high = ADCH;

- It is sufficient to read ADCH if 8 bit precision is required (ADLAR must be set).
  - (ADCH contains Most Significant Bits)

ADC Interrupts

- A very useful feature of the ADC in is that a CPU interrupt can be sent when ADC conversion completes:
  - If the ADC duration time is the slow link in your program, you can do other things whilst this goes on and it will remind you when it's done;
  - Alternatively, if you wish to repeatedly convert an analog input variable, and then perform some processing on this variable, such processing can be included in the ADC Interrupt Service Routine (ISR).

ADC Interrupts

- To enable ADC Interrupts, set bit 3 (ADIE) of ADCSRA.

	7	6	5	4	3	2	1	0
ADCSRA	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0

Ext I/O Registers